

Rummikub AI Agent 設計實作：從規則驗證到策略優化

課程：2026 資訊之芽 C++ 班・大作業二 期末報告

姓名：藍宥欣 組別：線上語法班

繳交日期：2026/07/04

一、專案簡介

本專案是 2026 資訊之芽 C++ 班大作業二的實作，主題為拉密（Rummikub）遊戲系統與 AI Agent 設計。專案目標不只是讓程式能夠正確判斷牌組是否合法，也需要設計一支能夠自動分析局勢、出牌、利用桌面牌重組，並嘗試打敗 baseline agent 的 AI。

在這個專案中，我完成了遊戲規則驗證、Human Agent 檔案讀取，以及 AI Agent 的策略設計。AI Agent 的核心目標是：在一般對戰中保留彈性與爆發力，在殘局中降低手牌失分，並且盡量避免提交非法盤面。這個專案讓我第一次完整體驗到「遊戲規則 → 程式建模 → AI 策略 → 壓力測試 → 策略修正」的開發流程。

二、遊戲基本功能實作

遊戲引擎 (validator.cpp)

實作了 `isValidRun` 與 `isValidGroup` 兩個核心驗證函式：

- Run (順組)**：3 張以上、同色、數字連續，Joker 可代入缺號位置
- Group (群組)**：3~4 張、同數字、顏色互異，Joker 可補缺色

出牌的唯一入口是 `Board::applyProposedSets()`，會依序檢查七件事：牌源合法性、有無重複、原桌面牌是否全部保留、未破冰前不可重組、每個 set 是否合法、破冰分數是否 ≥ 30 。只要有一項不通過，整批提案會被打回，桌面與手牌完全不會被改動。

檔案讀取 (human_agent.cpp)

完成 `waitForActionFile()` 輪詢等待 `action.json`，以及 `applyActionFile()` 解析並提交人類玩家的操作。

三、AI Agent 策略設計

`AIAgent_0` 的決策邏輯依優先序分成五層：

階段	核心機制	對應程式碼
破冰	兩層貪心：先只用純手牌找候選，湊不到 30 分才把 Joker 納入；候選依「Joker 用量少 → 分數高」排序	<code>tryInitialMeld()</code>
反僵局	牌堆剩餘 ≤ 20 張時放棄囤牌、全力進攻	<code>playTurn()</code>
囤牌觀察	手牌出現「重疊牌型」（同數字 ≥ 2 色、且鄰近數字也在手上）時，冷卻 4 回合保留戰術彈性	<code>internalCheckOverlap()</code>
反擊	監控對手連續抽牌次數，達門檻（實測後由 3 調整為 8）視為對手卡關，觸發全域重組	<code>enemy_draw_count</code>

核心創新：「大風吹」全域重組

這是本次實作中投入最多心力的部分（對應 `tryExtendBoard()`）。這個機制能不能成功重組，關鍵完全取決於「掃描」這一步做得對不對——掃描找出的必須是真正合法的最長 Run，重組才有意義；如果掃描邏輯有誤（例如分堆分錯顏色、Joker 用錯位置），產生的桌面就不會通過驗證，會被完整攔下、整批退回原狀。下圖是完整流程：

STEP 1 • 拆解

桌面 + 手牌全部攤開

依顏色分成四堆（紅／藍／黃／黑），Joker 另外抽出集中管理；桌面上原本就合法的 Group 整組跳過、不拆進顏色堆——避免拆散後湊不回去。



STEP 2 • 掃描

逐色掃描，拼出最長合法 Run

每個顏色堆由小到大排序後逐一掃描：數字連續就收進同一組；遇到缺口且缺口張數 $\leq$ 剩餘 Joker 數，就借 Joker 動態補上；缺口太大則收尾、另開新組（`checkRunsWithJokers()`）。這一步掃描得越準確、越完整，後面能重組出的牌組就越長、留給對手的餘地就越少。



STEP 3 • 優化

長龍切斷（Run ≥ 6 張時）

模擬每一個切點，計算切完之後手牌能接上頭尾的張數，取「能接最多牌」的切點，把長龍切成兩段，讓手牌有更多機會銜接上去。



STEP 4 • 驗證

安全回滾鎖（Safety Rollback Lock）

提交前逐一檢查：原桌面每一張牌是否都完整保留在新的合法組合中？桌面總張數是否確實比重組前增加？兩項全部成立才真正提交；只要有一項不符合，代表前面掃描或切斷有瑕疵，立刻整批 rollback，絕不把不合法盤面交給引擎判斷。

為什麼「掃描的方法有效，重組才有效」：STEP 4 的驗證完全是被動的——它不會「修正」錯誤，只會「攔下」錯誤。真正決定重組會不會成功的，是 STEP 2 掃描邏輯本身夠不夠正確、夠不夠完整。也因此，每一次大風吹成功送出，本身就是掃描邏輯正確性的直接證明；反之若掃描出的 Run 有誤（例如誤判連續性、Joker 補錯位置），STEP 4 會立刻攔下整批提案，該回合等同沒有動作。

開發過程中原本規劃用遞迴窮舉（`solveRummikub`）來找真正的最佳解，但考量到需要疊加安全回滾鎖來確保正確性，改用「迭代＋貪心規則」整套寫進 `tryExtendBoard`。`solveRummikub` 保留了函式介面，但目前未實際使用。

四、開發過程中的困難與解決

- Visualizer 讀檔格式不符：**把整場對局紀錄（陣列）直接覆蓋 `state.json`（預期是單一物件），導致 `state.players` 是 `undefined`、畫面報錯。拆成 `state.json`（當前局面）+ `state_history.json`（整場歷史）兩個檔案，對應 Replay Mode 的用途後解決。
- 反擊門檻太敏感：**`enemy_draw_count` 門檻原本設 ≥ 3 ，實測發現對手只要連續抽牌 3 次就幾乎每輪都觸發大風吹，反而干擾正常節奏，調高到 ≥ 8 後才穩定，只在對手真正卡關時出手。
- 本地一度測不到真正的 baseline：**`CMakeLists.txt` 裡負責把 `prebuilt/ai_agent_baseline0.cpp.o` / `ai_agent_baseline1.cpp.o` 串進執行檔的區塊被誤註解掉，導致連結失敗；為了先能編譯，`main.cpp` 一度暫時把 b0/b1 token 都導向自己的 `AIAgent_0` 頂替。後來把 `CMakeLists.txt` 那段還原、並在 Docker（`--platform=linux/amd64`，因為 `prebuilt` 物件檔是 x86_64 格式，跟 Apple Silicon 原生環境不相容）環境下重新編譯，才真正連結上助教提供的 baseline，跑出第五節的真實對戰數據。

五、測試結果

修正 `CMakeLists.txt` 中被誤註解掉的 baseline 連結區塊、並在 Docker（`--platform=linux/amd64`）環境下重新編譯，確認真正連結到助教提供的 `prebuilt/ai_agent_baseline0.cpp.o` 與 `ai_agent_baseline1.cpp.o` 後，進行了兩輪測試：

大樣本驗證（各 500 場）



對 baseline1 的勝率沒有到 100%，符合預期——baseline1 本身會出手牌中的完整牌組，比完全不出牌的 baseline0 更有威脅性；但在千場等級的大樣本下仍維持 64% 的整體優勢，排除了小樣本測試可能只是運氣好的疑慮。

逐場明細（各 5 場，取自大樣本測試前的驗證階段）



對局	AI_0_1 分數	對手分數	結果
vs baseline0 #1	67	394	勝 (+327)
vs baseline0 #2	78	408	勝 (+330)
vs baseline0 #3	71	465	勝 (+394)
vs baseline0 #4	139	450	勝 (+311)
vs baseline0 #5	137	441	勝 (+304)
vs baseline1 #1	89	211	勝 (+122)
vs baseline1 #2	119	154	勝 (+35)

vs baseline1 #3	139	216	勝 (+77)
vs baseline1 #4	33	92	勝 (+59)
vs baseline1 #5	71	137	勝 (+66)

分數為手牌點數總和，越低越好。對 baseline0 的平均勝場分差達 +333，對 baseline1 平均勝場分差 +72——分差較小的原因是 baseline1 本身會出手牌中的完整牌組，比完全不出牌的 baseline0 更有威脅性，但兩者最終都沒能撐過 AIAgent_0 的大風吹重組與殘局壓分能力。

驗證過程說明：本地端最初因 CMakeLists.txt 中串接 baseline 物件檔的區塊被誤註解掉，導致編譯時 b0／b1 token 一度改為暫時建立 AIAgent_0 本身（即先前版本報告中的「本地鏡像測試」）。修正 CMakeLists.txt 並在 Docker（linux/amd64 平台）環境重新編譯、確認連結到真正的 prebuilt/*.cpp.o 後，才產出上表這組真實對戰數據。

六、心得

最一開始的版本邏輯很單純：只要能破冰就無腦出牌，藉此把手牌壓到最低。但實際測試後發現，這樣的策略太依賴「找到當下最大化的組合」，牌堆快見底時還在猶豫要不要湊更好的組合，反而常常演變成雙方一直抽牌的僵局。

為了打破這個情況，我把判斷依據從「手牌怎麼打分數最高」改成「牌堆還剩多少張」：當牌堆數量低於門檻時，就不再追求最大化，先求有牌出。這個調整讓 agent 終於能穩定推進牌局，而不是卡在互相抽牌。

但光靠這個還是太看運氣。於是我開始把桌面上已經打出去的牌也一起納入考慮，設計出「一條龍」式的最長 Run 掃描：不只看手牌能不能湊出組合，也會嘗試把手牌接到桌面現有 Run 的頭尾，甚至把整個桌面拆開重拼成更長的連續數字，這也是後來「大風吹」全域重組的雛形。

在對戰 baseline1 的過程中，我發現一味進攻其實不夠聰明——對手的行為本身就是資訊。我加入了「觀察對手連續抽牌次數」的機制：如果對手連續好幾輪都在抽牌，代表牌堆裡已經沒有他能用的組合，這時候正是發動大風吹重組、搶進度的最佳時機（門檻經過實測從 3 調整到 8，因為 3 太敏感、幾乎每回合都觸發）。

實作大風吹的過程中，我原本規劃寫一個遞迴窮舉版本（`solveRummikub`），理論上能找到真正的最佳解。但後來考量到需要加上「安全回滾鎖」——重組完一定要驗證原本桌面上的牌是否還完整保留、張數是否真的增加，否則整批打回原狀——這種需要層層驗證的邏輯，用迭代加貪心規則反而更好掌控每一步的正確性，所以最後把整套邏輯改寫進

`tryExtendBoard`，`solveRummikub` 保留了介面但沒有真的實作。

補充說明：這份心得原本還包含兩個構想——「依手牌 Joker 張數（0/1/2 張）切換三種判斷分支」，以及「刻意出錯留數字空格、藉此引誘對手洩漏手牌」的戰術。誠實記錄一下：這兩個機制在最終版的 `ai_agent_0.cpp` 裡**沒有**實際寫成程式碼。目前的 Joker 處理是在 `tryInitialMeld()` 用「候選組合依 Joker 用量少、分數高排序」的統一貪心邏輯，而不是依手牌 Joker 數切換的三支；囤牌偵測則是 `internalCheckOverlap()`，判斷的是手牌本身的重疊牌型（同數字 ≥ 2 色、且鄰近數字也在手上），跟主動誘敵完全無關。留下這段是想保留當時的思考方向，即使最後沒有真的做出來。

七、結論

這次做出的是一支混合戰術型 `AI Agent_0`：

- **破冰階段：**兩層貪心排序，優先省下 Joker
- **中局：**依牌堆張數與手牌重疊牌型，在囤牌與進攻間切換
- **反擊時機：**抓到對手連續抽牌，立刻用大風吹反擊
- **殘局：**解除所有限制，迴圈出牌直到打不出為止

學到最深的一課是「先求穩定合法，再談最佳化」——安全回滾鎖多次擋下了不合法的重組提案。未來想把 `solveRummikub` 補成真正的遞迴／動態規劃搜尋，取代現在的貪心啟發式。

八、是否有外部協助（Discussion & AI Policy）

依課程規範揭露本次開發過程中使用 AI（Claude）協助的部分：

Claude 協助的範圍：

- Debug：協助定位 visualizer 讀取 `state.json` 時的格式不符問題（陣列 vs 單一物件），以及 `CMakeLists.txt` 中 baseline 連結區塊被誤註解、跨架構（arm64/x86_64）編譯導致的連結失敗問題
- 內容校對：將心得與報告文字敘述與實際送交的 `ai_agent_0.cpp` 逐項對照，指出敘述與程式碼不一致之處（即上方「補充說明」提到的兩處）

- 文件整理：協助整理與潤飾簡報、口說逐字稿與本報告的文字排版

未經 Claude 協助、完全由本人設計與撰寫的部分：

- `validator.cpp` 的 `isValidRun` / `isValidGroup` 驗證邏輯
- `ai_agent_0.cpp` 的所有策略與演算法設計，包括兩層貪心破冰、大風吹全域重組、長龍切斷優化、安全回滾鎖、重疊牌型偵測、對手抽牌監控反擊機制
- `human_agent.cpp` 的檔案讀取實作

本人對繳交的每一行程式碼負責，並理解其運作原理，如講師需要進行 Code Review，可隨時安排。